Feature List

Every feature described with user stories, acceptance criteria and limits

DeFi Contract Auditor (working name) | Draft v0.1 | October 25, 2026
Priorities: P0 = MVP essential, P1 = credible product, P2 = differentiator
Status: specification stage; nothing is built yet

How to read this document

Each feature has an ID, a priority, a release phase, a one-paragraph summary, user stories in the form 'As a [user], I want [capability], so that [benefit]', acceptance criteria that can be tested, and notes on limits or edge cases. Personas: **Founder** (early-stage developer or founder), **Auditor** (independent security reviewer), and **Manager** (compliance manager, later phase).

Feature overview

ID	Feature	Priority	Phase
F0	Projects, upload and audit lifecycle	P0	MVP
F1	Privilege Map ('Who can rug me?')	P0	MVP
F1.1	Role-holder declaration	P0	MVP
F1.2	Score explanation and mitigation detection	P0	MVP
F2	Static security findings	P0	MVP
F3	Coverage statement and disclaimers	P0	MVP
F4	Reports and export	P1	MVP / M8
F5	Accounts, teams and privacy controls	P0/P1	MVP / M8
F6	Evidence from the security library (retrieval)	P1	Phase 2
F7	Unverified observations panel	P1	Phase 2
F8	Verified patches and diff review	P1	Phase 2
F9	Prove-it exploit tests	P2	Phase 3
F10	Invariant Hunter Lite	P2	Phase 3
F11	API keys and CI integration	P1	M8

MVP features

F0 Projects, upload and audit lifecycle

Priority	Phase
P0	MVP

Users organize work into projects, upload Solidity files or a zipped Foundry/Hardhat project, start an audit, watch live progress, and can cancel or permanently delete it.

User stories

- *As a Founder, I want to upload a single file or a zip of my project, so that I can start a check without any setup.*
- *As a Founder, I want to see live progress by stage, so that I know the analysis is working and how long it may take.*
- *As a Founder, I want to cancel a running audit or delete a finished one, so that I stay in control of my code.*
- *As an Auditor, I want to organize audits by project, so that I can compare runs over time.*

Acceptance criteria

- Upload limit is configurable (default 10 MB); zip files are validated (no path traversal, symlinks or oversized archives).
- Creating an audit returns immediately (under 500 ms); work runs in the background.
- A live stream shows each stage starting and finishing and ends with a clear completed or failed event.
- If the code fails to compile, the user sees the compiler's file, line and message, not a blank error.
- Deleting an audit immediately purges the stored source and derived data.

Notes and limits

- Mixed compiler versions, missing imports, duplicate file names in different folders, empty projects and non-Solidity uploads must each give a clear message.
- If the compile succeeds but a downstream tool crashes, the audit degrades gracefully and records the limitation in the coverage statement.

F1 Privilege Map ('Who can rug me?')

Priority	Phase
P0	MVP (flagship)

For every privileged action in the contract, the tool answers three questions: who can do it, what is the worst case, and what limits abuse. Results are shown as a graph (roles on the left, powers on the right) and as a table, with exact code lines and a plain-English worst-case sentence per role. It is computed deterministically from the code, so it does not depend on an AI model's guess.

User stories

- *As a Founder, I want to see which roles can mint, burn, pause, upgrade, freeze or withdraw funds, so that I can reduce centralization risk before launch.*
- *As a Founder, I want a worst-case sentence per role in plain English, so that I can explain the risk to investors and non-technical teammates.*
- *As an Auditor, I want every power linked to the exact function and lines, so that I can verify it in seconds.*

- *As a Founder, I want to see when a role can grant other roles, so that I notice indirect escalation paths (the admin of a role can effectively do what that role can do).*
- *As a Founder, I want clear labels showing what is assumed versus declared, so that I do not over-trust or under-trust the result.*

Acceptance criteria

- Detects access control built with owner patterns, OpenZeppelin AccessControl roles, custom modifiers, and proxy admin / upgrade authorizers on the test fixtures.
- Every privileged action is assigned to one category in a fixed list: self-destruct or arbitrary delegatecall, upgrade, withdraw funds, mint, change oracle, burn others, blacklist or freeze, grant role, change fee, change parameter, pause, other privileged.
- Role risk uses role-admin escalation: a role that administers a critical role scores at least 95% of that role's score.
- Results are deterministic: the same input always gives identical output and ordering.
- A section titled 'What this view cannot see' lists blind spots that were hit (for example inline assembly, or a role assigned through an external contract).
- High and critical unmitigated results also generate candidate findings with direct evidence.

Notes and limits

- Unguarded functions that change privileged state are not shown in the map; they raise a separate 'missing access control' finding.
- When classification is unclear the tool uses 'other privileged' instead of guessing.
- Inherited and overridden modifiers, internal helper checks, aliasing of the caller address, and proxies without a supplied implementation are known edge cases covered by tests and noted in coverage.

F1.1 Role-holder declaration

Priority	Phase
P0	MVP

An optional panel lets the user say who actually holds each role: a single wallet, a multisig (with threshold and signer count), a timelock plus multisig (with delay), a renounced role, or an unknown contract. The risk score updates instantly. If nothing is declared, the tool assumes a single key and says so.

User stories

- *As a Founder, I want to declare that my owner role is a 3-of-5 multisig with a 48-hour timelock, so that the risk rating reflects my real setup.*
- *As a Founder, I want the result to say 'Holder not declared: assuming single key' when I skip this, so that I know the rating is pessimistic by design.*
- *As an Auditor, I want declarations clearly marked as user-asserted, so that the report never presents them as verified facts.*

Acceptance criteria

- Declaration is optional; the pessimistic default applies when missing.
- Changing a declaration recomputes the map in under 300 ms for a 2,000-line project.
- Validation: threshold cannot exceed signers; unknown roles are rejected.
- Reports show declared holders and mark them 'user-asserted'.

Notes and limits

- 'Renounced' is user-asserted and marked unverified. The tool does not connect to any blockchain, so it cannot confirm real wallets or ownership.

F1.2 Score explanation and mitigation detection

Priority	Phase
P0	MVP

The tool detects limits on powers (caps, built-in delays, two-step ownership transfer, reversibility) and shows how each factor changed the score in a 'How is this scored?' drawer.

User stories

- *As a Founder, I want to see which mitigations the tool found, so that I know what already protects me and what to add.*
- *As an Auditor, I want the score broken down by factor, so that I can challenge or confirm it.*
- *As a Founder, I want to see what each mitigation is worth, so that I can prioritize the cheapest effective fix, such as adding a delay.*

Acceptance criteria

- Each action shows base score and multipliers for cap, delay, reversibility and holder.
- Scoring weights are shown in the app and in every export, and live in one versioned configuration.
- Detected delays are not double-counted with a declared timelock (the stronger one applies).

F2 Static security findings

Priority	Phase
P0	MVP

Runs a curated set of standard detectors plus custom detectors (for example unprotected privileged writes and unbounded minting) and converts results into findings with severity, confidence, exact lines and at least one piece of direct evidence.

User stories

- *As a Founder, I want findings ranked by severity with exact lines, so that I fix the most dangerous issues first.*
- *As an Auditor, I want to dismiss a false positive and have it excluded from scoring, so that my triage persists.*
- *As a Founder, I want library code and test mocks excluded by default, so that I am not buried in noise.*

Acceptance criteria

- Every finding has a title, severity, confidence, one or more code spans, a description and at least one evidence record (enforced in the database).
- Dismissed findings stay dismissed and are excluded from the score.
- Duplicate findings from inheritance are merged.
- Dependency and mock folders are excluded by default, with a toggle to include them.

Notes and limits

- Findings from model text alone are never scored (see F7).

F3 Coverage statement and disclaimers

Priority	Phase
P0	MVP (cross-cutting)

Every report shows what was analyzed, what could not be analyzed and why, and what is out of scope for any code tool (economic design, oracle trust, governance, off-chain components, deployment configuration, key custody). A standard disclaimer appears everywhere.

User stories

- *As a reader of a report, I want to see what was not checked, so that I do not mistake silence for safety.*
- *As a Founder, I want the score labeled a heuristic risk indicator, so that I do not present it as a certification.*
- *As an Auditor, I want the toolchain and corpus versions stamped on every report, so that results are reproducible.*

Acceptance criteria

- The coverage card sits above the score in the interface and near the top of exports.
- The disclaimer is the same single text on screen, in the API and in every export.
- The words 'safe', 'secure', 'audited', 'certified' and 'compliant' are never used as verdicts.

F4 Reports and export

Priority	Phase
P1	MVP / M8

Download results as JSON (full data), Markdown (human-readable), SARIF (for developer tooling) and, later, PDF.

User stories

- *As a Founder, I want a readable report to share with my team, so that fixes can be assigned.*
- *As an Auditor, I want SARIF output, so that findings appear in my existing tools.*
- *As a Founder, I want a short audit-readiness summary, so that I can hand it to the human auditors I hire.*

Acceptance criteria

- All formats include disclaimer, coverage statement, privilege map summary, scoring weights, toolchain and corpus versions.
- Exports are only available when an audit is complete.
- SARIF output validates against the published schema.

F5 Accounts, teams and privacy controls

Priority	Phase
P0 / P1	MVP / M8

Sign-in through a standard identity provider, organization-level isolation of all data, adjustable retention, and permanent deletion.

User stories

- *As a Founder, I want my team to share projects while other organizations can never see them, so that my unreleased code stays private.*

- *As a Founder, I want to choose how long my code is kept (down to 1 hour), so that I can match my own risk comfort.*
- *As a Founder, I want a clear statement of what is sent to AI providers, so that I can make an informed choice.*

Acceptance criteria

- Every query is scoped to the organization and enforced by database row-level security; an automated test proves cross-tenant access fails.
- Default retention is 24 hours; deletion is immediate on request and is verified.
- Logs contain identifiers, hashes and timings only; an automated test checks logs for leaked source text.

Phase 2 features

F6 Evidence from the security library (retrieval)

Priority	Phase
P1	Phase 2

A curated library of public vulnerability references (security weakness registry entries and well-documented incidents) is searched for each finding. The finding shows up to three evidence cards with title, link, an exact quote and a relevance score. Only the tool's own summaries and short permitted excerpts are stored, with links to the originals.

User stories

- *As a Founder, I want to see a real past incident that matches a finding, so that I understand why it matters and trust the flag.*
- *As an Auditor, I want exact quotes and links, so that I can read the source myself.*
- *As a Founder, I want the tool to say 'no historical precedent retrieved' when nothing matches, so that I am not shown a stretched citation.*

Acceptance criteria

- A quote is shown only if it is an exact substring of the stored source text (checked by code, not by an AI).
- Results below a calibrated relevance threshold are dropped.
- Retrieval recall at 5 is at least 0.85 on a hand-labeled set.
- If the retrieval service is down, the audit continues and the coverage statement notes it.

Notes and limits

- Customer code is never added to the shared library. Searches use abstract, code-free descriptions of the issue.
- Each source's license is respected; no bulk copying of copyrighted reports.

F7 Unverified observations panel

Priority	Phase
P1	Phase 2

Model-generated hypotheses that no deterministic tool confirmed appear in a separate, clearly labeled, unscored list, capped at low display severity.

User stories

- *As an Auditor, I want leads that the tool is not sure about kept separate, so that they help my review without polluting the confirmed findings.*
- *As a Founder, I want unverified items clearly labeled, so that I do not panic over, or ignore, them wrongly.*

Acceptance criteria

- Items are never counted in the score.
- The panel is collapsed by default with a plain-language explanation.
- Each item names the code it refers to and is checked against the code's structure before display.

F8 Verified patches and diff review

Priority	Phase
P1	Phase 2

For a finding, the tool proposes a code fix as a unified diff. A fix is shown only if it was applied to a scratch copy, compiled, re-scanned (the targeted problem is gone and no new high or critical issues appear) and, where tests exist, measured for gas change. Users can edit the diff and re-verify.

User stories

- *As a Founder, I want a ready-made fix, so that I can apply it in minutes.*
- *As a Founder, I want proof that the fix compiles and clears the issue, so that I do not trade one problem for another.*
- *As an Auditor, I want to edit a proposed patch and re-run the checks, so that I can adapt it to the project's style.*
- *As a Founder, I want to see the gas cost change of a fix, so that security does not silently make my contract expensive.*

Acceptance criteria

- Patches are offered only when status is verified.
- For upgradeable contracts, any fix that changes storage layout is blocked with an explanation.
- Up to three attempts are made; if none verify, the user gets guidance text and no patch.
- The interface shows badges for compiles, finding cleared, no new findings, tests, and gas (or 'gas not measured').
- Patch success rate on supported classes is at least 70% in evaluation.

Notes and limits

- The user's own tests and scripts are not executed. Only the tool's compile and analysis steps run in the sandbox.
- Conflicting patches for two findings are applied sequentially in a scratch copy and conflicts are reported.

Phase 3 features

F9 Prove-it exploit tests

Priority	Phase
P2	Phase 3

For supported finding types (such as reentrancy and missing access control), the tool writes a test that demonstrates the problem. The test must fail on the original code and pass on the verified patch; only then is the finding marked 'Proven' with higher confidence.

User stories

- *As a Founder, I want a runnable demonstration of a bug, so that I can see it is real and not a false alarm.*
- *As an Auditor, I want to read and rerun the test, so that I can use it in my report.*
- *As a Founder, I want the same test to pass after my fix, so that I know the fix works.*

Acceptance criteria

- A proof is shown only when it fails on the original and passes on the patched code, checked twice to rule out flaky results.
- The test must assert an attacker gain or a broken rule, not just run without error.
- Tests run only in a network-less sandbox with filesystem and external-call features disabled.
- Tests are not included in default exports and are rate-limited.

Notes and limits

- The feature works only against the user's own submitted code; it is not an exploit-writing tool for other people's contracts.

F10 Invariant Hunter Lite

Priority	Phase
P2	Phase 3

The tool classifies the contract type (token, vault, staking, lending) and runs a library of human-written 'rules that must always hold' against it, using automated random testing. A broken rule comes with the exact sequence of calls that breaks it.

User stories

- *As a Founder, I want standard safety rules for my contract type checked automatically, so that I catch logic bugs that pattern scanners miss.*
- *As an Auditor, I want a concrete call sequence for each violation, so that I can reproduce it.*
- *As a Founder, I want to confirm or correct the detected contract type, so that the right rules are applied.*

Acceptance criteria

- Initial rule sets: token (supply equals sum of balances, no unauthorized mint), vault (no round trip gains, no withdrawing more than owned), staking (rewards claimed do not exceed rewards funded), lending (debt stays within collateral limits).
- Each rule reports passed, violated, error or skipped, and every 'skipped' has a reason.
- Known-buggy fixtures are flagged violated and known-good fixtures pass.
- Runs are time-bounded.

Notes and limits

- Rules are written and reviewed by humans in this phase. Model-inferred rules are a later feature.

Later features

F11 API keys and CI integration

Priority	Phase
P1	M8

Scoped API keys let teams run audits from their build pipeline and download results automatically.

User stories

- *As a Founder, I want to run the check on every release branch, so that new risks are caught before merging.*
- *As an Auditor, I want to script batches of audits, so that I can process client repositories efficiently.*

Acceptance criteria

- Keys are shown once, stored only as hashes, scoped, revocable and rate-limited.
- A documented sequence (upload, create audit, poll, export SARIF) works from a shell script.

Deferred features (not scheduled)

ID	Feature	Why deferred
D1	Regulatory mechanics flags and counsel question lists	Requires a reviewed regulatory corpus. Rules in some key jurisdictions, including India, are still unsettled, and many obligations concern companies, not code. Output would be questions for counsel, never violation verdicts.
D2	Jurisdiction toggle	Depends on D1.
D3	Full Invariant Hunter (model-inferred rules with human confirm and edit)	Higher build cost and noise risk; needs the Lite version's data first.
D4	Deployment-configuration input (constructor arguments, initial roles)	Many real risks depend on how a contract is deployed; valuable but adds scope.
D5	Pull-request diff audits and living risk history	Best after the core flow and CI are proven.
D6	Multi-contract and composability analysis	Large complexity increase.
D7	Other languages and chains	Auditor supply and tooling differ by ecosystem.
D8	Self-hosted deployment	Enterprise need; build after the hosted product is stable.

> Out of scope permanently: post-launch monitoring, wallet tracking and on-chain transaction analysis. This product is a pre-deployment advisory tool only.